

AI ASSISTED CODING

ASSIGNMENT -10.1

B.GEETHANJALI

2303A51005

Task Description #1 – Syntax and Logic Errors

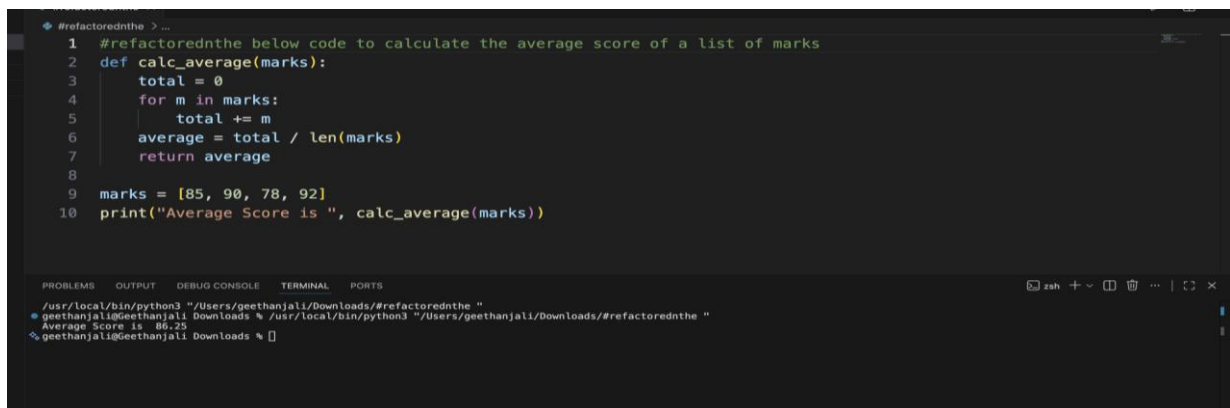
Task: Use AI to identify and fix syntax and logic errors in a faulty Python script.

Sample Input Code:

```
# Calculate average score of a student
def calc_average(marks):
    total = 0
    for m in marks:
        total += m
    average = total / len(marks)
    return avrage # Typo here
marks = [85, 90, 78, 92]
print("Average Score is ", calc_average(marks))
```

Expected Output:

- Corrected and runnable Python code with explanations of the Fixes.

A screenshot of a code editor with a dark theme. The editor shows a Python script that calculates the average of a list of marks. The code is as follows:

```
1 #refactorednthe below code to calculate the average score of a list of marks
2 def calc_average(marks):
3     total = 0
4     for m in marks:
5         total += m
6     average = total / len(marks)
7     return average
8
9 marks = [85, 90, 78, 92]
10 print("Average Score is ", calc_average(marks))
```

The editor has a terminal at the bottom. The terminal shows the command to run the script: `/usr/local/bin/python3 "/Users/geethanjali/Downloads/#refactorednthe "` and the output: `Average Score is 86.25`. The terminal also shows the prompt `geethanjali@geethanjali Downloads %`.

Task Description #2 – PEP 8 Compliance

Task: Use AI to refactor Python code to follow PEP 8 style guidelines.

Sample Input Code:

```
def area_of_rect(L,B): return L*B
print(area_of_rect(10,20))
```

Expected Output:

- Well-formatted PEP 8-compliant Python code.

```
#refactorednthe > ...
1 def area_of_rect(L,B) : return L*B
2 print(area_of_rect(10,20))
3 #refactorednthe above code to enhance readability and maintainability
4 #provide type hints for the function parameters and return type and include a docstring to explain the purpose
5 def area_of_rectangle(length: float, breadth: float) -> float:
6     """
7     Calculate the area of a rectangle.
8
9     Parameters:
10    length (float): The length of the rectangle.
11    breadth (float): The breadth of the rectangle.
12
13    Returns:
14    float: The area of the rectangle calculated as length multiplied by breadth.
15    """
16    return length * breadth
17 # Example usage
18 print(area_of_rectangle(10.0, 20.0))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

/usr/local/bin/python3 "/Users/geethanjali/Downloads/#refactorednthe "

geethanjali@geethanjali Downloads % /usr/local/bin/python3 "/Users/geethanjali/Downloads/#refactorednthe "

200

200.0

geethanjali@geethanjali Downloads %

Task Description #3 – Readability Enhancement

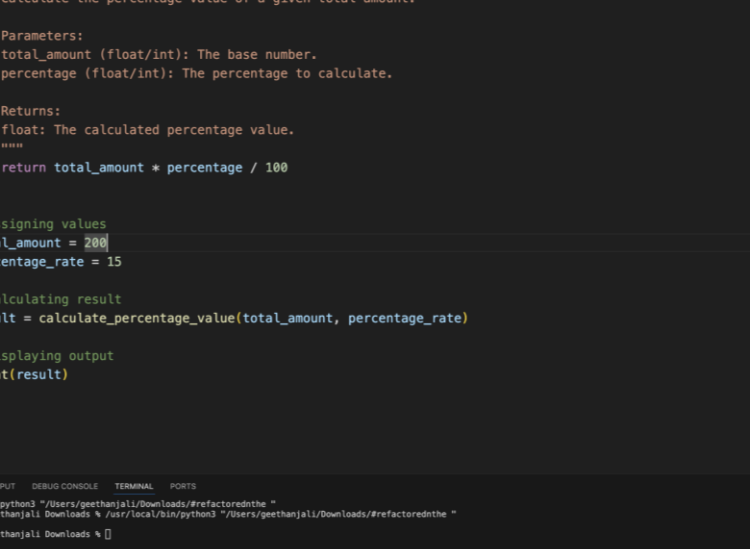
Task: Use AI to make code more readable without changing its logic.

Sample Input Code:

```
def c(x,y):
return x*y/100
a=200
b=15
print(c(a,b))
```

Expected Output:

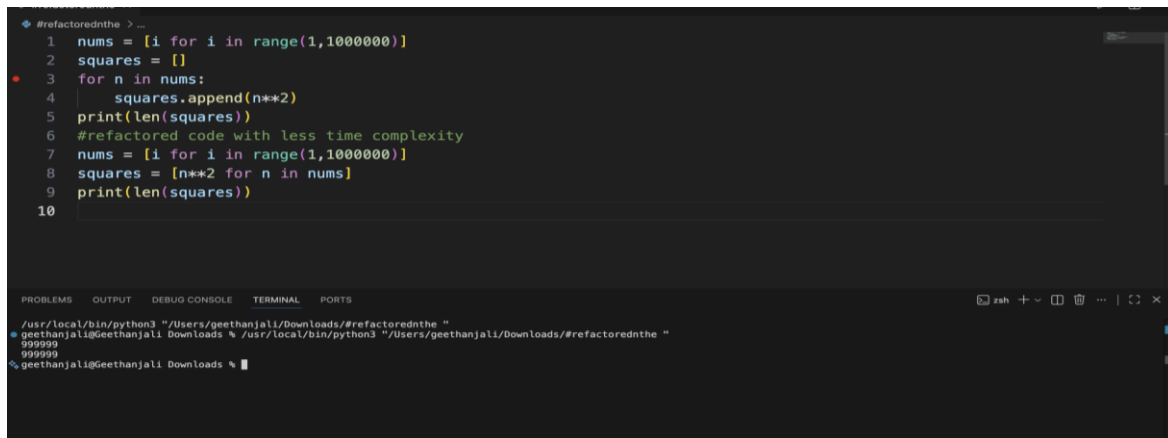
- Python code with descriptive variable names, inline comments, and clear formatting.



```
1 def calculate_percentage_value(total_amount, percentage):
2     """
3     Calculate the percentage value of a given total amount.
4
5     Parameters:
6     total_amount (float/int): The base number.
7     percentage (float/int): The percentage to calculate.
8
9     Returns:
10    float: The calculated percentage value.
11    """
12    return total_amount * percentage / 100
13
14 # Assigning values
15 total_amount = 200
16 percentage_rate = 15
17
18 # Calculating result
19 result = calculate_percentage_value(total_amount, percentage_rate)
20
21 # Displaying output
22 print(result)
```

```
geethanjali@geethanjali Downloads % python3 ~/Users/geethanjali/Downloads/#refactorednthe "
38.0
geethanjali@geethanjali Downloads %
```

nums = [i for i in range(1,1000000)] squares = [] for n in nums: squares.append(n**2)
print(len(squares)) Expected Output: • Optimized code using list comprehensions or
vectorized operations.



```
#refactorednthe > ...
1  nums = [i for i in range(1,1000000)]
2  squares = []
3  for n in nums:
4      squares.append(n**2)
5  print(len(squares))
6  #refactored code with less time complexity
7  nums = [i for i in range(1,1000000)]
8  squares = [n**2 for n in nums]
9  print(len(squares))
10
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

/usr/local/bin/python3 "/Users/geethanjali/Downloads/#refactorednthe "

geethanjali@geethanjali Downloads % /usr/local/bin/python3 "/Users/geethanjali/Downloads/#refactorednthe "

999999

999999

geethanjali@geethanjali Downloads %

Task Description #6 – Complexity Reduction

Task: Use AI to simplify overly complex logic.

Sample Input Code:

```
def grade(score):
```

```
    if score >= 90:
```

```
        return "A"
```

```
    else:
```

```
        if score >= 80:
```

```
            return "B"
```

```
        else:
```

```
            if score >= 70:
```

```
                return "C"
```

```
            else:
```

```
                if score >= 60:
```

```
                    return "D"
```

```
            else:
```

```
                return "F"
```

Expected Output:

- Cleaner logic using elif or dictionary mapping.



The screenshot shows a VS Code editor with a Python file named `refactorednth.py`. The code defines a `grade(score)` function that returns letter grades A, B, C, or D based on the score. Below the function, there are example usage lines. The terminal at the bottom shows the command `python3 refactorednth.py` being executed, and the output displays the grades for scores 95, 85, 75, 65, and 55.

```
#refactorednth.py
def grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "E"

# Example usage:
print(grade(95)) # Output: A
print(grade(85)) # Output: B
print(grade(75)) # Output: C
print(grade(65)) # Output: D
print(grade(55)) # Output: E
```

```
zsh
geethanjalig@geethanjalig Downloads: ~$ python3 refactorednth.py
A
B
C
D
E
geethanjalig@geethanjalig Downloads: ~$
```